

INFO145 - Diseño y Análisis de Algoritmos

Introducción a los Algoritmos Aproximados

Héctor Ferrada
Instituto de Informática
Universidad Austral de Chile

Introducción

Los algoritmos aproximados son técnicas utilizadas para encontrar soluciones cercanas a la óptima en problemas de optimización que son difíciles o imposibles de resolver exactamente en tiempo razonable, especialmente aquellos que son NP-hard. Dado que para muchos problemas NP-hard no se conocen algoritmos eficientes (de tiempo polinomial) que garanticen la solución óptima, los algoritmos aproximados ofrecen una alternativa práctica: encontrar soluciones que no son óptimas, pero sí suficientemente buenas, y hacerlo en tiempo polinomial.

A diferencia de las heurísticas, que pueden encontrar soluciones razonables sin garantías formales, los algoritmos aproximados proporcionan garantías matemáticas sobre la calidad de la solución y sobre el tiempo de ejecución. Esto significa que para cualquier instancia del problema, la solución entregada estará dentro de un factor conocido (llamado factor de aproximación o garantía α) respecto a la solución óptima.

Definición Formal

Un algoritmo A es un algoritmo de aproximación para un problema de optimización Π si, dada una instancia I de tamaño n , produce una solución S tal que:

- Para problemas de minimización:

$$\frac{C_A(I, n)}{C_{OPT}(I, n)} \leq \alpha$$

- Para problemas de maximización:

$$\frac{C_{OPT}(I, n)}{C_A(I, n)} \leq \alpha$$

donde $C_{OPT}(I, n)$ es el valor óptimo para la instancia I de tamaño n , y $C_A(I, n)$ es el valor obtenido por el algoritmo A para la misma instancia, y α es el factor de aproximación.

Ejemplo 1. Cobertura de Vértices (Vertex Cover)

Dado un grafo no dirigido $G = (V, E)$, un *vertex cover* (cobertura de vértices) es un subconjunto $C \subseteq V$ tal que para cada arista $(u, v) \in E$, al menos uno de sus extremos pertenece a C , es decir,

$$\forall (u, v) \in E, \quad u \in C \quad \text{o} \quad v \in C.$$

El problema de *Vertex Cover* consiste en encontrar un vertex cover de tamaño mínimo.

Este problema es conocido por ser NP-completo, por lo que no se conoce un algoritmo eficiente para resolverlo exactamente en todos los casos.

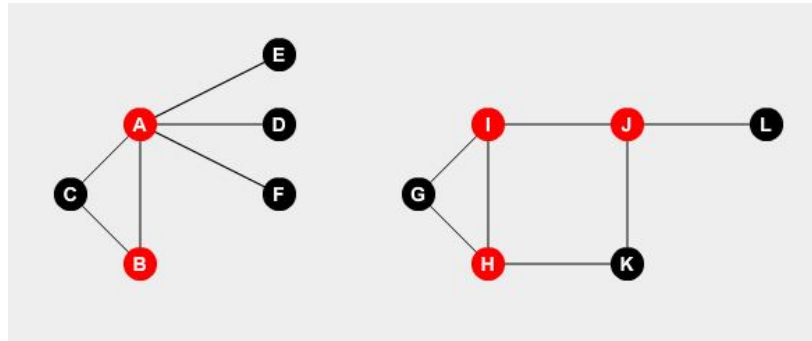


Figure 1: Ejemplo Vertex Cover

Solución Exhaustiva o Fuerza Bruta (Exacta)

El algoritmo exhaustivo prueba todos los subconjuntos posibles de vértices y selecciona el de menor tamaño que cubre todas las aristas.

```
VertexCoverExhaustivo( $G=(V,E)$ ):  
    min_cover = V // peor caso: todos los vértices  
    for cada subconjunto C de V:  
        if C cubre todas las aristas de E:  
            if  $|C| < |min\_cover|$ :  
                min_cover = C  
    return min_cover
```

Complejidad temporal: $O(2^{|V|} \cdot |E|)$, ya que se exploran todos los subconjuntos y se verifica la cobertura para cada uno.

Solución Aproximada 1 (por arista aleatoria)

Este algoritmo construye una cobertura de vértices C a partir de un emparejamiento maximal M en el grafo $G = (V, E)$. La idea es seleccionar repetidamente una arista cualquiera, añadir ambos extremos a la solución y eliminar todas las aristas cubiertas.

Complejidad

- La selección y eliminación de aristas se realiza en tiempo $O(|E|)$.
- Por lo tanto, el algoritmo es eficiente y práctico para grafos grandes.

Algorithm 1 APROXIMACION_VERTEX_COVER($G = (V, E)$)

```
1:  $C \leftarrow \emptyset$ 
2:  $E' \leftarrow E$ 
3: while  $E' \neq \emptyset$  do
4:   Seleccionar una arista arbitraria  $(u, v) \in E'$ 
5:    $C \leftarrow C \cup \{u, v\}$ 
6:   Eliminar de  $E'$  todas las aristas incidentes a  $u$  o  $v$ 
7: end while
8: return  $C$ 
```

Análisis

- La solución C es un vertex cover porque cada arista seleccionada está cubierta por al menos uno de sus extremos.
- El conjunto de aristas seleccionadas forma un emparejamiento maximal¹ M .
- Cualquier vertex cover óptimo C^* debe incluir al menos un vértice de cada arista en M , por lo que $|C^*| \geq |M|$.
- Como C incluye ambos extremos de cada arista en M , entonces $|C| = 2|M| \leq 2|C^*|$.
- Por lo tanto, el algoritmo es una aproximación de factor 2 para el problema de Vertex Cover.
- **En resumen:**

Aspecto	Solución Exhaustiva	Solución Aproximada
Método	Exhaustivo o Fuerza bruta	Selección iterativa de aristas
Complejidad temporal	$O(2^{ V } \cdot E)$	$O(E)$
Calidad de solución	Óptima	Factor 2 de aproximación
Aplicabilidad	Grafos pequeños	Grafos grandes y prácticos

Solución Aproximada 2 (por grado máximo)

Este algoritmo construye una cobertura de vértices C seleccionando iterativamente el vértice de grado máximo hasta cubrir todas las aristas.

1. **Inicialización:** Se comienza con un conjunto vacío C , que representará la cobertura de vértices.
2. **Selección iterativa:** Mientras existan aristas sin cubrir en el grafo:

¹Un emparejamiento maximal en grafos es un conjunto de aristas (conexiones entre pares de vértices) tal que ninguna otra arista del grafo puede añadirse a ese conjunto sin que dos aristas compartan un vértice. Es decir, si intentas agregar cualquier otra arista al emparejamiento, ya no cumpliría la condición de que las aristas no tengan vértices en común

- Se elige un vértice v con el *grado máximo*, es decir, el vértice que tiene el mayor número de aristas incidentes.
- Se añade v al conjunto C .
- Se elimina v y todas las aristas incidentes a v del grafo, ya que estas aristas quedan cubiertas.

3. **Terminación:** El proceso se repite hasta que no queden aristas por cubrir.

4. **Resultado:** El conjunto C es devuelto como una cobertura aproximada de vértices.

Algorithm 2 APROXIMACION_VERTEX_COVER_GRADO_MAX($G = (V, E)$)

```

1:  $C \leftarrow \emptyset$ 
2:  $G' \leftarrow G$                                 ▷ Copia del grafo original
3: while  $E(G') \neq \emptyset$  do                    ▷ Mientras haya aristas en  $G'$ 
4:    $v \leftarrow$  Vértice de grado máximo en  $G'$ 
5:    $C \leftarrow C \cup \{v\}$ 
6:   Eliminar  $v$  y todas las aristas incidentes a  $v$  de  $G'$ 
7: end while
8: return  $C$ 

```

El factor de aproximación de este algoritmo es $O(\log n)$, donde n es el número de vértices en el grafo. A diferencia del algoritmo que selecciona aristas arbitrarias, este no garantiza un factor constante de 2.

Comentarios

- Este algoritmo es una heurística voraz.
- En la práctica, a menudo funciona bien, pero puede dar soluciones subóptimas.
- La complejidad temporal depende de cómo se implementen las operaciones de selección y eliminación, pero típicamente es polinomial.

Ejemplo 2. Set Cover (Cobertura de Conjuntos)

Problema: Dados un universo U y una colección de subconjuntos $S_1, \dots, S_m \subseteq U$, encontrar el menor número de subconjuntos cuya unión cubra U ; es decir, se busca encontrar el subconjunto más pequeño de una colección de conjuntos que cubra todos los elementos del universo.

Algoritmo Exhaustivo

Note que la Complejidad temporal es $O(n \cdot 2^m)$, ya que prueba todas las combinaciones posibles de subconjuntos y debe verificar en $O(n)$ si $U = \mathcal{X}$.

Algorithm 3 Algoritmo Exhaustivo para Minimum Set Cover (mínimo garantizado)

Require: El universo $\mathcal{X}$ y la familia de subconjuntos $\mathcal{F}$ cuya union es $\mathcal{X}$

Ensure: Subcolección mínima $C \subseteq \mathcal{F}$ que cubre $\mathcal{X}$

```
1:  $C = \mathcal{X}$ 
2: for cada combinación  $C'$  de  $\mathcal{F}$  do
3:    $U \leftarrow \bigcup_{S_i \in C'} S_i$ 
4:   if  $U = \mathcal{X} \wedge |C'| < |C|$  then       $\triangleright$  Verificar cobertura completa y menor a la actual
5:      $C = C'$                               $\triangleright$  Nueva solución mínima encontrada
6:   end if
7: end for
8: return  $C$ 
```

Algoritmo Greedy

En cada paso, seleccionar el conjunto que cubra el mayor número de elementos aún no cubiertos.

Algorithm 4 GREEDY-SETCOVER tomado de [Cormen y otros, 2009]

Require: El universo $\mathcal{X}$ y la familia de subconjuntos $\mathcal{F}$ cuya union es $\mathcal{X}$

```
1: procedure GREEDY-SETCOVER( $\mathcal{X}, \mathcal{F}$ )
2:    $\mathcal{U} = \mathcal{X}$ 
3:    $\mathcal{C} = \emptyset$ 
4:   while  $\mathcal{U} \neq \emptyset$  do                                 $\triangleright$  tiempo  $O(\min\{|\mathcal{X}|, |\mathcal{F}|\})$ 
5:     seleccionar  $S \in \mathcal{F}$  que maximice  $|\mathcal{U} \cap S|$          $\triangleright$  tiempo  $O(|\mathcal{U}| \cdot |\mathcal{F}|)$ 
6:      $\mathcal{U} = \mathcal{U} - S$ 
7:      $\mathcal{C} = \mathcal{C} \cup S$ 
8:   end while
9:   return  $\mathcal{C}$ 
10: end procedure
```

Complejidad:

- En cada iteración, se debe encontrar el conjunto que cubre la mayor cantidad de elementos no cubiertos. Esto puede costar hasta $O(n \times m)$, donde n es el número de conjuntos y m el tamaño del universo.
- En el peor caso, se pueden seleccionar hasta n conjuntos, por lo que la complejidad total es aproximadamente

$$O(n^2 \times m).$$

Este algoritmo es eficiente para instancias grandes y ofrece una solución aproximada con un factor de aproximación de $H(m)$ (el número armónico de m), es decir, aproximadamente $\ln m + 1$.

Explicación del Factor de Aproximación

El algoritmo selecciona en cada paso el conjunto que cubre la mayor cantidad de elementos no cubiertos hasta ese momento. Entonces:

- Sea U el universo de elementos a cubrir y $s = |U|$.
- Definimos el **número armónico** $H(s)$ como:

$$H(s) = \sum_{k=1}^s \frac{1}{k} \leq \ln s + 1,$$

donde $\ln$ es el logaritmo natural.

- En cada iteración, el algoritmo voraz elige un conjunto que cubre al menos una fracción proporcional de los elementos restantes por cubrir. Esto implica que el número de elementos no cubiertos decrece rápidamente.
- Formalmente, si en una etapa quedan r elementos sin cubrir, el conjunto elegido cubre al menos $\frac{r}{k}$ elementos, donde k es el tamaño del conjunto óptimo mínimo (o una cota relacionada).
- Al sumar el costo amortizado de cubrir cada elemento, se obtiene que el costo total del algoritmo greedy es a lo sumo $H(s)$ veces el costo óptimo.
- Este análisis muestra que el algoritmo greedy garantiza una solución con un factor de aproximación de:

$$H(s) \approx \ln s + 1.$$

- Además, se ha demostrado que no es posible aproximar el problema de Set Cover en tiempo polinomial con un factor mejor que $(1 - o(1)) \ln s$, a menos que $P = NP$.

En resumen, el factor de aproximación $H(s)$ surge del comportamiento decreciente del número de elementos sin cubrir en cada paso y del uso del número armónico para amortiguar el costo total en relación con la solución óptima.

Demostración del Factor de Aproximación $O(\log n)$ para el Algoritmo Greedy por Grado Máximo del Vertex Cover

El problema de *Vertex Cover* puede ser formulado como un problema de *Set Cover*, donde:

- Los elementos a cubrir son las aristas E del grafo $G = (V, E)$.
- Cada vértice $v \in V$ representa un conjunto que cubre las aristas incidentes a v .

El algoritmo voraz que selecciona en cada paso el vértice de mayor grado (es decir, el conjunto que cubre más aristas no cubiertas) es análogo al algoritmo voraz clásico para Set Cover.

Sea $n = |V|$ el número de vértices del grafo.

1. En el problema de Set Cover, el algoritmo voraz que en cada iteración escoge el conjunto que cubre la mayor cantidad de elementos sin cubrir garantiza un factor de aproximación

$$H(d) = \sum_{i=1}^d \frac{1}{i} \leq \log d + 1,$$

donde d es el tamaño máximo de cualquier conjunto.

2. En nuestro caso, el tamaño máximo de un conjunto corresponde al grado máximo Δ del grafo, es decir, el máximo número de aristas incidentes a un vértice.
3. Por lo tanto, el factor de aproximación del algoritmo voraz para Vertex Cover basado en grado máximo es

$$H(\Delta) \leq \log \Delta + 1.$$

4. Dado que $\Delta \leq n - 1$, se concluye que el factor de aproximación es

$$O(\log n).$$

De este modo, el algoritmo voraz que selecciona en cada paso el vértice de grado máximo para cubrir aristas no cubiertas es una heurística con factor de aproximación $O(\log n)$. Esto significa que el tamaño del vertex cover obtenido puede ser hasta un factor logarítmico mayor que el tamaño mínimo posible.

Ejemplo 3. TSP Métrico (Problema del Viajante de Comercio Métrico)

El Problema del Viajante de Comercio (TSP, por sus siglas en inglés: Travelling Salesman Problem) puede definirse formalmente de la siguiente manera:

Dado:

- Un grafo completo $G = (V, E)$, donde V es un conjunto de n vértices (ciudades) y E es un conjunto de aristas que conectan cada par de vértices.
- Una función de costo $c : E \rightarrow \mathbb{R}$ que asigna un costo (o distancia) $c(e)$ a cada arista $e \in E$.

Objetivo: Encontrar un ciclo Hamiltoniano (una ruta que visita cada vértice exactamente una vez y regresa al vértice inicial) de costo mínimo. Formalmente, encontrar una permutación π de los vértices $\{1, 2, \dots, n\}$ que minimice la función objetivo:

$$\min_{\pi} \sum_{i=1}^{n-1} c(v_{\pi(i)}, v_{\pi(i+1)}) + c(v_{\pi(n)}, v_{\pi(1)}),$$

donde:

- $v_{\pi(i)}$ es el i -ésimo vértice en la permutación π .

- $c(v_{\pi(i)}, v_{\pi(i+1)})$ es el costo de la arista entre el vértice $v_{\pi(i)}$ y el vértice $v_{\pi(i+1)}$.
- $c(v_{\pi(n)}, v_{\pi(1)})$ es el costo de la arista que cierra el ciclo, regresando al vértice inicial.

El objetivo es encontrar la permutación π^* que minimice esta suma total, es decir:

$$\pi^* = \arg \min_{\pi} \sum_{i=1}^{n-1} c(v_{\pi(i)}, v_{\pi(i+1)}) + c(v_{\pi(n)}, v_{\pi(1)}).$$

Algoritmo exhaustivo (fuerza bruta)

El algoritmo exhaustivo para resolver el problema del viajante de comercio (TSP) consiste en explorar todas las posibles rutas que visitan cada ciudad exactamente una vez y regresan a la ciudad inicial, con el fin de encontrar aquella con el costo total mínimo.

El procedimiento es el siguiente:

1. Se fija una ciudad inicial para evitar contar rutas equivalentes por rotación.
2. Se generan todas las permutaciones posibles de las ciudades restantes.
3. Para cada permutación, se calcula la suma de los costos de viajar entre ciudades consecutivas en el recorrido, incluyendo el regreso a la ciudad inicial.
4. Se mantiene un registro del recorrido con menor costo encontrado hasta el momento.
5. Al finalizar la exploración de todas las permutaciones, se devuelve la ruta óptima y su costo total.

Este método garantiza encontrar la solución óptima, pero su complejidad factorial $O(n!)$ limita su uso a instancias pequeñas.

Algoritmo de Aproximación (Christofides):

El algoritmo de Christofides construye un ciclo hamiltoniano con costo a lo sumo 1.5 veces el óptimo, con los pasos:

1. Construir un árbol de expansión mínima (MST) del grafo.
2. Encontrar el conjunto de vértices de grado impar en el MST.
3. Encontrar un emparejamiento perfecto mínimo en el subgrafo inducido por esos vértices.
4. Unir el MST y el emparejamiento para formar un multigrafo euleriano.
5. Encontrar un circuito euleriano en ese multigrafo.
6. Construir un ciclo hamiltoniano haciendo un recorrido de ese circuito euleriano omitiendo vértices repetidos (shortcutting).

Algorithm 5 Algoritmo de Christofides para TSP

Require: Grafo completo $G = (V, E)$ con matriz de distancias M que cumple la desigualdad triangular

Ensure: Ciclo hamiltoniano aproximado H

- 1: Construir un árbol de expansión mínima T de G
 - 2: Identificar el conjunto $O \subseteq V$ de vértices con grado impar en T
 - 3: Calcular un emparejamiento perfecto mínimo M en el subgrafo inducido por O
 - 4: Construir el multigrafo $G' = T \cup M$, que es euleriano
 - 5: Encontrar un circuito euleriano C en G'
 - 6: Construir H recorriendo C y eliminando vértices repetidos (shortcutting)
 - 7: **return** H
-

Notas sobre complejidad y aproximación

- La construcción del árbol de expansión mínima se realiza en $O(n^2)$ para grafos densos.
- El emparejamiento perfecto mínimo se calcula en $O(n^3)$ usando el algoritmo de Edmonds.
- Encontrar el circuito euleriano y realizar el shortcutting se hacen en tiempo lineal.
- El ciclo H cumple la cota de aproximación:

$$c(H) \leq \frac{3}{2}c(H^*)$$

donde $c(H^*)$ es el costo del ciclo óptimo.

Emparejamiento Perfecto Mínimo

Sea $G = (V, E)$ un grafo no dirigido, donde V es el conjunto de vértices y E el conjunto de aristas.

- Un **emparejamiento** $M \subseteq E$ es un conjunto de aristas tal que ninguna dos aristas en M comparten un vértice, es decir, para todo $u \in V$, el grado de u en el subgrafo $G(M) = (V, M)$ es a lo sumo 1.
- Un **emparejamiento perfecto** es un emparejamiento M que cubre todos los vértices de G , es decir, cada vértice $v \in V$ está incidente a exactamente una arista en M :

$$\deg_{G(M)}(v) = 1, \quad \forall v \in V.$$

- Si el grafo es ponderado con una función de costos $c : E \rightarrow \mathbb{R}^+$, el **emparejamiento perfecto mínimo** es aquel emparejamiento perfecto M^* que minimiza la suma total de los costos de sus aristas:

$$M^* = \arg \min_{M \text{ emparejamiento perfecto}} \sum_{e \in M} c(e).$$

Nota: La resolución eficiente del problema de emparejamiento perfecto mínimo en grafos generales se realiza mediante algoritmos como el algoritmo Blossom de Edmonds, con complejidad polinomial $O(n^3)$.

Demostración del factor de aproximación 1.5:

Sea c^* el coste óptimo. El árbol de recubrimiento mínimo T tiene coste $c(T) \leq c^*$, ya que cualquier ciclo hamiltoniano es un árbol de recubrimiento más una arista. El emparejamiento mínimo M tiene coste $c(M) \leq \frac{1}{2}c^*$ (por propiedades del TSP métrico). Por lo tanto, el recorrido euleriano tiene coste $c(T) + c(M) \leq c^* + \frac{1}{2}c^* = \frac{3}{2}c^*$. Al recorrer E y saltar repeticiones, el coste no aumenta por la desigualdad triangular, así que el ciclo resultante tiene coste a lo sumo $1.5c^*$:

$$c(\text{solución}) \leq c(T) + c(M) \leq 1.5c^*$$

Por lo tanto, el algoritmo es una 1.5-aproximación.

Revise ejemplo del Cormen, de cómo se transforma el MST en un ciclohamiltoniano.

Importancia y Uso

Los algoritmos aproximados son fundamentales en la práctica porque permiten resolver problemas complejos en tiempos razonables, con garantías formales sobre la calidad de la solución. Son especialmente útiles en áreas como la logística, redes, diseño de circuitos, y más, donde los problemas suelen ser NP-C o NP-hard y las soluciones exactas son inviables para grandes instancias.